Assignment 9.2 - Designing a Modern, AI-Augmented DevOps Workflow

Date: October 10, 2026
Student: Jonathan Cantu
Professor: Richard Krasso
Course: WEB430

Introduction

SilverLine Education needs a development workflow that supports frequent releases without compromising reliability, security, or the sensitive information entrusted to its platform. I recommend a GitHub-centered DevSecOps approach combining short-lived development branches, automated CI/CD pipelines, AI-assisted development, and risk-based governance. GitLab could support a comparable design using its native CI/CD capabilities. The objective is not to automate every decision, but to establish a predictable system where automation improves Flow and Feedback while human oversight remains proportional to risk. This reflects Krasso's (2026) emphasis on integrating governance, compliance, and change management directly into modern development workflows.

PART I

1. Version Control and Branching Strategy

I recommend a trunk-based development strategy using a protected `main` branch and short-lived feature branches. Development should begin with appropriately scoped JIRA stories or defect tickets, each identifying acceptance criteria, dependencies, and testing expectations. Developers would create focused branches, complete manageable changes, and merge through pull requests (PRs).

This approach avoids the integration problems associated with long-lived branches. Smaller changes are easier to understand, test, review, and reverse. Developers also receive faster feedback because code integrates frequently rather than accumulating differences over several weeks.

Branch protection would prohibit direct pushes to `main`, require successful automated checks, and enforce appropriate reviews. Release tags would identify deployable versions without maintaining unnecessary permanent release branches. Feature flags would allow incomplete functionality to remain disabled until ready, separating deployment from feature availability.

2. Pull Requests, Reviews, and Quality Gates

Every PR should reference its JIRA ticket and include a description of the change, testing evidence, potential risks, and rollback considerations. PRs should remain small enough for meaningful review, with larger initiatives divided into independently testable increments.

GitHub Actions would execute automated quality gates, including formatting, linting, unit tests, dependency scanning, and security checks. Prek could run repository-defined pre-commit checks locally and within CI, while Snyk would identify vulnerable dependencies and other supported security issues. GitHub Copilot could provide an additional review perspective, identifying possible defects or missing tests.

However, AI-assisted review should supplement rather than replace qualified human reviewers. Code owners would review sensitive components, and changes affecting authentication, grading calculations, student information, or deployment controls would receive additional scrutiny. Required checks must pass before merging, with documented exceptions reserved for authorized personnel.

This structure makes quality a consistent part of development rather than an unpredictable final inspection.

3. CI/CD Pipeline Design

SilverLine's pipeline should provide a repeatable path from source code to production.

When a PR opens or changes, GitHub Actions would retrieve the source, install pinned dependencies using reproducible commands such as `npm ci`, and execute formatting, static analysis, unit tests, and security scans. Integration tests would validate interactions with databases and external services. Automated builds would then produce versioned, immutable application artifacts.

After an approved merge, the pipeline would deploy the same tested artifact to a staging environment configured to resemble production. Staging validation would include API integration tests, critical user journeys, authentication checks, and performance smoke tests. Infrastructure-as-code validation would verify environment configuration before changes are applied.

Production deployments should use progressive delivery, such as canary releases or controlled rolling updates, supported by health checks and automated monitoring. Feature flags would allow functionality to be enabled gradually. Initially, production promotion would require human approval, with low-risk, well-established release categories becoming candidates for further automation as confidence increases.

Every deployment should retain its artifact version, source commit, approval record, and release status. Database migrations require special consideration: backward-compatible schema changes should precede dependent application changes, and recovery plans should account for migrations that cannot simply be reversed.

These practices reduce release risk by making changes smaller, observable, and recoverable.

4. AI-Assisted Development

AI tools can reduce repetitive engineering work throughout SilverLine's development lifecycle. GitHub Copilot could assist with boilerplate code, documentation, refactoring suggestions, and initial test-case generation. Developers would remain responsible for validating generated code against requirements, architectural standards, and existing behavior.

AI-generated tests could identify overlooked edge cases, particularly around grading calculations, missing assignments, date boundaries, and access permissions. However, developers must independently evaluate whether those tests represent correct business requirements rather than merely confirming the AI's assumptions.

AI could also analyze pipeline execution history to identify recurring failures, slow tests, or unusual build times. During incidents, AI-assisted monitoring could summarize logs, correlate alerts, and suggest possible causes. These outputs would remain recommendations, not independently authorized production actions.

Responsible AI use requires explicit boundaries. SilverLine should prohibit submitting identifiable student records, credentials, or other sensitive information to unapproved AI services. AI-generated changes must pass the same tests, reviews, and security gates as human-written changes. Significant AI assistance should be disclosed in PR documentation, with accountability assigned to the developer accepting the output.

5. Security, Compliance, and Governance

Security must be incorporated throughout development rather than postponed until deployment. Developers should follow secure coding practices covering input validation, authentication, authorization, error handling, and dependency management. Threat modeling should identify risks involving student records, grading integrity, account access, and third-party integrations.

Secrets must never be hardcoded in repositories. GitHub Actions environments and an approved secrets manager would provide protected credentials. Where supported, short-lived credentials through OpenID Connect would reduce reliance on persistent cloud access keys.

Access controls should follow least privilege, with multifactor authentication, protected environments, and separation of duties for sensitive changes. Production services should generate centralized logs, metrics, and alerts without exposing student information or secrets.

Because SilverLine serves educational institutions, compliance planning should account for applicable student privacy obligations, including FERPA where relevant, contractual requirements, and documented data retention policies. Compliance controls should produce evidence automatically through PR histories, test results, security findings, deployment approvals, and audit logs.

Governance policies must also define approved AI tools, permitted data classifications, acceptable automated actions, and human approval requirements. AI-generated recommendations should be

traceable and reviewable. Periodic evaluations should assess tool effectiveness, accuracy, security, and possible bias.

As Krasso (2026) explains, governance should be integrated into existing workflows rather than operating as a separate administrative obstacle.

6. Change Management in Automated Systems

I recommend categorizing changes according to potential impact, reversibility, and confidence in the available validation.

Low-risk changes, such as documentation updates or minor fixes with strong automated coverage, could follow standardized approval and deployment paths. Moderate-risk changes would require normal peer review and staging verification. High-risk changes, including authentication modifications, grading logic, infrastructure permissions, or major database migrations, would require designated technical or security approval and explicit recovery plans.

AI-generated changes would be classified by their potential consequences, not simply by whether AI was involved. A minor AI-generated documentation correction differs substantially from an AI-generated authorization change.

Every exception or emergency override should identify who authorized it, why it was necessary, and what follow-up review is required. Production releases should have defined rollback procedures, tested backups, and observable health criteria. Feature flags, artifact redeployment, and controlled traffic shifting would support recovery where appropriate.

This approach follows the Week 9 principle that change management should enable safe delivery rather than impose identical approval burdens on every change (Krasso, 2026).

7. Continuous Improvement

SilverLine should treat its workflow as a system that evolves through evidence rather than assumptions. Teams should establish baseline measurements for lead time, deployment frequency, change failure rate, recovery time, PR review delays, and pipeline reliability.

Retrospectives would examine where work accumulates, where feedback arrives too late, and which controls generate unnecessary noise. Instead of making sweeping changes, teams should conduct targeted experiments.

For example, if integration tests create a significant bottleneck, the team might separate fast PR checks from broader staging validation while preserving appropriate coverage. Results should be measured against both delivery speed and escaped defects.

AI-assisted analysis could help identify recurring patterns, but engineers must validate its conclusions before changing the workflow. Improvement should focus on the actual system constraint rather than simply making individual activities faster.

8. Connection to the Three Ways

The proposed workflow supports Gene Kim's Three Ways of DevOps through Flow, Feedback, and Continuous Learning.

Flow improves through small JIRA stories, short-lived branches, focused PRs, repeatable pipelines, and progressive releases.

Feedback improves through automated testing, Prek and Snyk checks, human and AI-assisted reviews, deployment health checks, and production monitoring.

Continuous Learning develops through retrospectives, incident reviews, measured experiments, and periodic reassessment of AI tools and governance policies.

These principles reinforce one another. Faster delivery without reliable feedback increases risk, while excessive controls without risk-based judgment create bottlenecks. The objective is a balanced system that continuously improves its ability to deliver trustworthy software.

Conclusion

SilverLine Education should adopt a practical, governed DevSecOps workflow in which automation handles repeatable activities, AI assists engineering analysis, and humans retain accountability for consequential decisions. By combining GitHub Actions, Prek, Snyk, Copilot, protected deployments, traceable approvals, and continuous measurement, SilverLine can improve delivery performance while protecting students and maintaining organizational trust.

The most important outcome is not maximum automation, but a reliable development system capable of adapting safely as technology, requirements, and organizational risks evolve.

References

- GitHub. “Using secrets in GitHub Actions”. GitHub Docs. https://docs.github.com/en/actions/concepts/security/secrets
- Krasso, Richard (2026). “WEB 430 weekly reading”. Bellevue University.
- Microsoft. “Security in DevOps (DevSecOps)”. Microsoft Learn. https://learn.microsoft.com/en-us/devops/operate/security-in-devops
- Snyk. “Snyk GitHub Actions”. GitHub. https://github.com/snyk/actions

PART II

For this part of the assignment, you will create a small, functional demonstration of how AI-assisted tools support your workflow. You will work with a code snippet, a pipeline snippet, and an AI-assisted tool to generate:

- Documentation
- Test cases
- Code improvements
- Pipeline explanation
- Security recommendations
- Compliance or governance observations

This mirrors the kinds of tasks AI tools would perform in your proposed workflow.

Code Snippet (JavaScript):

```
function calculateGrade(scores) {
  if (!Array.isArray(scores) || scores.length === 0) {
    return null;
  }

  const total = scores.reduce((sum, s) => sum + s, 0);

  const average = total / scores.length;

  if (average >= 90) return "A";

  if (average >= 80) return "B";

  if (average >= 70) return "C";

  if (average >= 60) return "D";

  return "F";
}
```

Pipeline Snippet (YAML)

```
name: GradeService CI

on:
  push:
    branches: [ "main0" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Install dependencies
        run: npm install
```

```
      - name: Run tests
        run: npm test
```

Steps:

1. Copy the code snippet into an AI-assisted tool.
2. Ask the tool to generate documentation.
3. Ask the tool to generate test cases.
4. Ask the tool to suggest improvements.
5. Copy the pipeline snippet into the tool.
6. Ask the tool to explain the pipeline.
7. Ask the tool to identify risks or missing steps.
8. Ask the tool to suggest improvements.
9. Review all output and reflect on what you notice.
10. As the AI to identify any governance, compliance, or change-management considerations.

AI Response

SilverLine Education DevSecOps Project Analysis

1. AI-Generated Documentation 📝

JSDoc Documentation

```
/**
 * Calculates a letter grade based on an array of numerical scores
 *
 * @function calculateGrade
 * @param {number[]} scores - Array of numerical scores (0-100 scale)
 * @returns {string|null} Letter grade ("A"-"F") or null for invalid input
 *
 * @example
 * calculateGrade([85, 90, 95]); // Returns "A"
 * calculateGrade([65, 70, 75]); // Returns "C"
 * calculateGrade([]);           // Returns null
 * calculateGrade("invalid");    // Returns null
 *
 * @description
```

```
* This function calculates an average from an array of scores and returns
* a letter grade based on the following thresholds:
* - A: 90 ≤ average ≤ 100
* - B: 80 ≤ average < 90
* - C: 70 ≤ average < 80
* - D: 60 ≤ average < 70
* - F: average < 60
*
* @throws {Error} Does not throw errors, returns null for invalid inputs
* @since Version 1.0
*/
```

Function Analysis

- **Purpose**: Converts numerical scores to letter grades using standard academic thresholds
- **Parameters**: Single array parameter `scores` expected to contain numbers
- **Expected Input**: Array of numbers representing percentage scores (0-100)
- **Return Values**: Letter grades "A" through "F" or `null` for invalid input
- **Grading Thresholds**: Standard 10-point intervals starting at 90 for "A"
- **Assumptions**:
  - Scores are on a 0-100 percentage scale
  - All scores are equally weighted
  - Missing scores are not handled (empty arrays return null)
  - Negative scores are not prevented
  - Scores above 100 are not prevented
- **Limitations**:
  - No rounding logic for decimal averages
  - No validation of individual score ranges
  - No handling of extra credit or weighted assignments
  - Returns null for empty arrays instead of a default grade
  - No support for +/- grading (A-, B+, etc.)

2. AI-Generated Test Cases 🧪

Existing Behavior Tests (Current Implementation)

```
describe('calculateGrade - Existing Behavior', () => {
  test('returns A for average >= 90', () => {
    expect(calculateGrade([95, 90, 92])).toBe('A');
    expect(calculateGrade([90, 90, 90])).toBe('A');
  });
```

```
test('returns B for average >= 80 and < 90', () => {
  expect(calculateGrade([85, 82, 88])).toBe('B');
  expect(calculateGrade([80, 80, 80])).toBe('B');
});

test('returns C for average >= 70 and < 80', () => {
  expect(calculateGrade([75, 72, 78])).toBe('C');
  expect(calculateGrade([70, 70, 70])).toBe('C');
});

test('returns D for average >= 60 and < 70', () => {
  expect(calculateGrade([65, 62, 68])).toBe('D');
  expect(calculateGrade([60, 60, 60])).toBe('D');
});

test('returns F for average < 60', () => {
  expect(calculateGrade([55, 50, 58])).toBe('F');
  expect(calculateGrade([0, 0, 0])).toBe('F');
});

test('returns null for empty array', () => {
  expect(calculateGrade([])).toBe(null);
});

test('returns null for non-array input', () => {
  expect(calculateGrade(null)).toBe(null);
  expect(calculateGrade('test')).toBe(null);
  expect(calculateGrade(100)).toBe(null);
  expect(calculateGrade({})).toBe(null);
```

```
  });

  test('handles decimal averages without rounding', () => {
    expect(calculateGrade([89.9, 89.9, 89.9])).toBe('B');
    expect(calculateGrade([79.9, 79.9, 79.9])).toBe('C');
  });
});
```

Proposed Requirements Tests (Missing Features)

```
describe('calculateGrade - Proposed Requirements', () => {
  // Business rule clarifications needed
  test('should round averages to nearest integer', () => {
    // Requires business decision: Should 89.5 round to 90 (A) or 89 (B)?
    // expect(calculateGrade([89.5, 89.5, 89.5])).toBe('A');
  });

  test('should validate individual score ranges (0-100)', () => {
    // Should negative scores be rejected?
    // expect(calculateGrade([-10, 90, 80])).toThrow();

    // Should scores above 100 be rejected?
    // expect(calculateGrade([110, 90, 80])).toThrow();
  });

  test('should handle edge cases with single score', () => {
    expect(calculateGrade([95])).toBe('A');
    expect(calculateGrade([55])).toBe('F');
  });

  test('should handle large arrays efficiently', () => {
    const largeArray = Array(10000).fill(85);
```

```
  expect(calculateGrade(largeArray)).toBe('B');
});

test('should consider weighted scores', () => {
  // Requires API change: calculateGrade(scores, weights)
  // const scores = [90, 80, 70];
  // const weights = [0.5, 0.3, 0.2]; // Weighted average = 84
  // expect(calculateGrade(scores, weights)).toBe('B');
});

test('should support +/- grading', () => {
  // Requires business decision on thresholds
  // expect(calculateGrade([92, 94, 96])).toBe('A');
  // expect(calculateGrade([87, 88, 89])).toBe('B+');
  // expect(calculateGrade([82, 83, 84])).toBe('B');
  // expect(calculateGrade([78, 79, 79])).toBe('C+');
});
});
```

3. AI-Generated Code Improvements 🔧

Identified Issues

- **Input Validation Weaknesses**:
  - No validation of individual score values (negative, >100, NaN)
  - No type checking for array elements
  - No handling of string numbers (e.g., `["85", "90"]`)
- **Security Concerns**:
  - No protection against extremely large arrays causing memory issues
  - Potential for infinite loops if scores array contains non-numeric values
- **Maintainability Issues**:
  - Magic numbers (90, 80, etc.) not defined as constants
  - No error messages for invalid inputs
  - No logging for debugging
- **Business Logic Concerns**:
  - No rounding logic for decimal averages
  - No consideration for extra credit or weighted assignments
  - No support for different grading scales

Revised Implementation

```
// Configuration constants for maintainability
const GRADE_THRESHOLDS = {
  A: 90,
  B: 80,
  C: 70,
  D: 60,
  F: 0
};

const SCORE_RANGE = {
  MIN: 0,
  MAX: 100
};

/**
 * Calculates a letter grade with improved validation and error handling
 * @param {number[]} scores - Array of scores (0-100)
 * @returns {string} Letter grade or throws error for invalid input
 * @throws {TypeError} For invalid input types
 * @throws {RangeError} For scores outside 0-100 range
 */
function calculateGrade(scores) {
  // Enhanced validation
  if (!Array.isArray(scores)) {
    throw new TypeError('Input must be an array');
  }

  if (scores.length === 0) {
```

```
    throw new Error('Scores array cannot be empty');
  }

  // Validate each score
  const validatedScores = scores.map((score, index) => {
    const numScore = Number(score);

    if (isNaN(numScore)) {
      throw new TypeError(`Score at index ${index} is not a valid number`);
    }

    if (numScore < SCORE_RANGE.MIN || numScore > SCORE_RANGE.MAX) {
      throw new RangeError(
        `Score at index ${index} (${numScore}) must be between ` +
        `${SCORE_RANGE.MIN} and ${SCORE_RANGE.MAX}`
      );
    }

    return numScore;
  });

  // Calculate average with rounding (business decision: round to nearest integer)
  const total = validatedScores.reduce((sum, score) => sum + score, 0);
  const average = Math.round(total / validatedScores.length);

  // Determine grade
  if (average >= GRADE_THRESHOLDS.A) return "A";
  if (average >= GRADE_THRESHOLDS.B) return "B";
  if (average >= GRADE_THRESHOLDS.C) return "C";
  if (average >= GRADE_THRESHOLDS.D) return "D";
```

```
    return "F";
}
```

Changes Made & Business Decisions Needed

- **Changed**: Returns errors instead of null for invalid inputs (requires API consumer adjustment)
- **Changed**: Added rounding logic (requires business approval on rounding rules)
- **Changed**: Added strict validation (0-100 range)
- **Needs Clarification**:
  - Should we allow extra credit (scores > 100)?
  - Should we support different grading scales?
  - Should we add +/- grading?
  - Should scores be weighted differently?
  - What rounding method (floor, ceil, round, custom)?

4. AI-Generated Pipeline Explanation 

Pipeline Analysis

```
name: GradeService CI              # Workflow name for GitHub UI

on:

  push:

    branches: [ "main0" ]          # Typo: Should likely be "main"

  pull_request:

    branches: [ "main" ]           # Correct branch for PR triggers


jobs:

  build-and-test:                  # Single job named "build-and-test"

    runs-on: ubuntu-latest         # GitHub-hosted runner environment

    steps:

      - uses: actions/checkout@v3   # Checks out repository code


      - name: Install dependencies  # Human-readable step name

        run: npm install           # Installs Node.js dependencies


      - name: Run tests            # Human-readable step name

        run: npm test              # Executes test suite via npm
```

Execution Triggers

- **On Push**: When code is pushed to branch named `"main0"` (likely typo)
- **On Pull Request**: When PR is created targeting `"main"` branch

Step-by-Step Accomplishments

1. **Checkout**: Downloads repository code to runner workspace
2. **Install Dependencies**: Runs `npm install` to install package dependencies
3. **Run Tests**: Executes `npm test` to run automated tests

Current Limitations

- ❌ No build verification (compilation, bundling)
- ❌ No linting or code quality checks
- ❌ No security scanning
- ❌ No dependency vulnerability checks
- ❌ No caching for faster runs
- ❌ No artifact creation or deployment
- ❌ No matrix testing across Node.js versions
- ❌ No code coverage reporting
- ❌ No environment-specific configurations

5. AI-Generated Pipeline Risks and Missing Steps ⚠️

Critical Issues (Must Fix)

1. **Branch Typo**: `"main0"` instead of `"main"` means pushes to main branch won't trigger pipeline
2. **No Security Scanning**: Missing SAST, dependency scanning, secret detection
3. **No Permissions Configuration**: Runs with default permissions (potentially excessive)
4. **No Node.js Version Specification**: Uses system default, not project's supported versions
5. **No Dependency Locking**: `npm install` may install different versions on each run

Security Risks

- **Dependency Vulnerabilities**: No scanning for known vulnerabilities
- **Secrets Exposure**: No scanning for accidentally committed secrets
- **Code Injection**: No static analysis for security flaws
- **Supply Chain Attacks**: No validation of package integrity

Missing Quality Gates

- ❌ No linting (ESLint) enforcement
- ❌ No type checking (TypeScript)
- ❌ No code formatting (Prettier)
- ❌ No test coverage requirements
- ❌ No branch protection rules
- ❌ No required approvals for deployment

Unreliable Practices

- **Non-Reproducible Builds**: No package-lock.json usage guarantee
- **Floating Dependencies**: `npm install` without `--ci` or `--frozen-lockfile`

- **No Caching**: Reinstalls all dependencies every run
- **Single Environment**: Only tests on Ubuntu latest

Deployment Limitations

- No staging environment deployment
- No production deployment automation
- No rollback capabilities
- No infrastructure as code validation
- No database migration checks

6. AI-Generated Pipeline Improvements 🚀

Revised GitHub Actions Workflow

```
name: DevSecOps Pipeline

on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main", "develop" ]
  schedule:
    # Weekly security scan on Sundays
    - cron: '0 0 * * 0'

env:
  NODE_VERSION: '18'              # Define supported Node.js version
  REGISTRY: ghcr.io               # GitHub Container Registry

jobs:
  security-scan:
    name: Security Scanning
    runs-on: ubuntu-latest
    permissions:
      contents: read
```

```
    security-events: write
    actions: read
  steps:
    - uses: actions/checkout@v4

    - name: Run Snyk to check for vulnerabilities
      uses: snyk/actions/node@master
      env:
        SNYK_TOKEN: ${{ secrets.SNYK_TOKEN }}
      with:
        args: test --all-projects

    - name: Run CodeQL Analysis
      uses: github/codeql-action/analyze@v3
      with:
        languages: javascript

    - name: Secret Scanning
      uses: trufflesecurity/trufflehog@latest
      with:
        extra_args: --regex --entropy=False

  quality-checks:
    name: Quality Gates
    runs-on: ubuntu-latest
    needs: [security-scan]
    strategy:
      matrix:
        node-version: [16, 18, 20]  # Test across supported versions
    steps:
```

```
    - uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}
        cache: 'npm'

    - name: Install dependencies
      run: npm ci               # Reproducible install from lockfile

    - name: Lint code
      run: npm run lint || echo "Linting failed"

    - name: Type check (if TypeScript)
      run: |
        if [ -f "tsconfig.json" ]; then
          npx tsc --noEmit
        fi

    - name: Run tests with coverage
      run: npm test -- --coverage
      env:
        CI: true

    - name: Upload coverage
      uses: codecov/codecov-action@v4

  build-verify:
    name: Build Verification
```

```
  runs-on: ubuntu-latest
  needs: [quality-checks]
  steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${{ env.NODE_VERSION }}

    - name: Install dependencies
      run: npm ci

    - name: Build project
      run: npm run build

    - name: Verify build output
      run: |
        if [ -d "dist" ]; then
          echo "Build directory exists"
          ls -la dist/
        else
          echo "No build directory found"
        fi

  deploy-staging:
    name: Deploy to Staging
    runs-on: ubuntu-latest
    needs: [build-verify]
    if: github.ref == 'refs/heads/main' || github.event_name == 'pull_request'
```

```
    environment:
      name: staging
      url: https://staging.silverline-education.com
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to staging
        run: |
          # Add deployment commands here
          echo "Deploying to staging environment"
        env:
          DEPLOY_KEY: ${{ secrets.STAGING_DEPLOY_KEY }}
```

Integration Notes

- **Snyk Configuration**: Requires `SNYK_TOKEN` secret in repository settings
- **CodeQL**: Free for public repositories, available for private on GitHub Enterprise
- **TruffleHog**: Open-source secret scanner
- **Matrix Testing**: Tests across multiple Node.js versions for compatibility
- **Caching**: Uses GitHub's built-in caching for npm dependencies
- **Environment Deployments**: Separate staging environment with manual approval gates

Distinguishing Working vs. Illustrative Configurations

- ✅ **Working**: Node.js setup, npm caching, matrix testing
- ✅ **Working**: CodeQL analysis (native GitHub integration)
- ⚠️ **Requires Setup**: Snyk (requires account and token)
- ⚠️ **Requires Setup**: Deployment steps (specific to infrastructure)
- ⚠️ **Requires Setup**: Environment secrets and URLs

7. Governance, Compliance, and Change Management

Educational System Specific Considerations

Student Privacy (FERPA Compliance)

- Grade data constitutes educational records requiring protection
- Access must be limited to authorized personnel only
- Audit trails must track all grade modifications
- Data encryption required both in transit and at rest
- Minimum necessary data collection principles apply

Grading Integrity Requirements

- Immutable audit logs of all grade calculations
- Version control for grading algorithms

- Approval workflows for grade formula changes
- Tamper-evident logging for grade modifications
- Regular integrity checks and reconciliation

Access Control Framework

- Role-based access control (RBAC) with least privilege
- Multi-factor authentication for grade modifications
- Separation of duties between development and production access
- Regular access reviews and recertification
- Emergency break-glass procedures with enhanced logging

Traceability and Audit Requirements

- Git commit signatures for all changes
- Change tickets linked to code modifications
- Pre- and post-deployment verification checks
- Automated compliance documentation generation
- Regular third-party audit readiness assessments

AI-Generated Change Governance

- **AI-Permitted Recommendations**:
  - Code optimization suggestions
  - Test case generation
  - Documentation updates
  - Security scanning configuration
  - Pipeline efficiency improvements
- **Human Authorization Required**:
  - Changes to grading algorithms
  - Modifications to student data handling
  - Privacy or compliance-related changes
  - Production deployment approvals
  - Changes to audit logging mechanisms

Rollback Strategies

- Blue-green deployments for zero-downtime rollbacks
- Database migration versioning with forward/backward compatibility
- Feature flags for gradual rollouts
- Automated health checks before traffic shifting
- Documented rollback procedures with RTO/RPO targets

Change Approval Workflow

1. **Proposal**: AI or developer suggests change
2. **Review**: Security, compliance, and technical review
3. **Testing**: Staging environment validation
4. **Approval**: Human sign-off for production changes
5. **Deployment**: Automated with monitoring
6. **Verification**: Post-deployment validation
7. **Documentation**: Update compliance documentation

8. Limitations and Human Review Required 🧐

Assumptions Made

1. **Business Rules**: Standard 10-point grading scale without +/- grades
2. **Input Range**: Scores limited to 0-100 range
3. **Weighting**: Equal weighting of all scores

4. **Rounding**: Mathematical rounding to nearest integer
5. **Error Handling**: Throw errors rather than return null for invalid inputs

Unresolved Business Requirements

- **Grading Scale**: Should we support custom grading scales per course?
- **Extra Credit**: How to handle scores above 100%?
- **Missing Assignments**: How to calculate grades with incomplete work?
- **Weighted Assignments**: Different weights for tests vs homework?
- **Curving Grades**: Should class-wide adjustments be supported?
- **Grade Appeals**: Process for manual grade overrides?

Implementation Risks Requiring Review

1. **Breaking Changes**: Changing from null returns to thrown errors
2. **Performance**: No limits on array size could cause memory issues
3. **Precision**: Floating-point arithmetic could cause rounding errors
4. **Internationalization**: Hard-coded grade letters may need translation
5. **Accessibility**: No consideration for assistive technologies

Human Review Required Before Acceptance

- **Security Review**: Validate input validation and error handling
- **Compliance Review**: Ensure FERPA and data protection compliance
- **Business Review**: Confirm grading algorithm meets academic requirements
- **Performance Review**: Validate handling of edge cases and large datasets
- **Accessibility Review**: Ensure grade display meets accessibility standards
- **Legal Review**: Terms of service and data handling agreements

Testing Gaps Identified

- Integration testing with student database
- Performance testing with realistic dataset sizes
- Security penetration testing
- Disaster recovery testing
- Compliance audit simulation

Recommended Next Steps

1. **Business Requirements Gathering**: Clarify grading policies with stakeholders
2. **Security Assessment**: Formal security review of proposed changes
3. **Compliance Check**: Legal review of data handling practices
4. **User Acceptance Testing**: Validate with actual educators
5. **Performance Testing**: Load test with realistic student volumes
6. **Documentation Update**: Update API documentation reflecting changes

Reflection on AI-Assisted DevOps

DeepSeek performed well in documenting the JavaScript grading function, generating initial test cases, and identifying weaknesses in the existing GitHub Actions pipeline. It correctly identified the `main0` branch typo, recommended more reproducible dependency installation using `npm ci`, and proposed additional security and quality checks. I also appreciated its recognition of FERPA considerations, grading integrity, auditability, and the need for human approval of sensitive changes.

However, closer examination revealed that several proposed tests were merely placeholders, and its revised pipeline allowed linting failures to pass without failing the build. More concerning was its decision to introduce automatic grade rounding and change the function's error-handling behavior without first obtaining approval for those business requirements. In an educational platform, even a seemingly minor adjustment to grading calculations could affect student outcomes and introduce compliance or governance concerns.

These findings reinforce why our proposed workflow combines AI-assisted development with GitHub Actions quality gates, automated security scanning, and human code reviews. AI-generated changes should be evaluated through automated testing, security validation, and risk-based approval before being merged or deployed. I found that DeepSeek understood the importance of governance conceptually but did not consistently apply those principles to its own recommendations.

This exercise reinforced my understanding that AI can accelerate development and improve feedback, but reliable DevOps practices still depend on human accountability, trustworthy validation, and disciplined change management.